

SZABIST University Larkana

Department of Computer Science

CSC3109 Software Engineering

Assignment 1
Developing your custom APIs to provide your Microservice

Instructor: Abdul Muqtadar

Objective:

- To understand API development
- Designing your own APIs
- Backend communicating with frontend using APIs

Submission:

- A .txt file which contains link to your public repository
- Your repository must have folder for each task
- Each folder in your repository must contain codes and video demonstration of TASK (or public link of video demonstration)

Task 1: Database-Driven Flask API with Streamlit Interface [1 Mark]

Scenario: A university wants to develop a student management application that stores student records in a database and allows users to manage them through a web interface.

Requirements (Flask API):

- Connect Flask to an SQLite database.
- Create a students table containing student ID, name, department, and email.
- Implement the following endpoints:
    - GET /students — retrieve all students.
    - GET /students/<id> — retrieve a specific student.
    - POST /students — add a new student.
    - PUT /students/<id> — update a student's information.
    - DELETE /students/<id> — delete a student.

Requirements (Streamlit):

- Display all students in a table.
- Provide a form to add a new student.
- Allow users to select and update an existing student.
- Allow users to delete a student.
- Refresh the displayed records after each successful operation.

Task 2: API Key Generation and Sharing [1 Mark]

Scenario: RootSoftware develops a text-processing API that other developers need to access. You are required to create an API and provide three API keys to three different consumers for testing.

Requirements:

- Develop a Flask API with a POST /process endpoint that accepts a text string in JSON format and returns the processed text. For example, it may return the text in uppercase.
- Generate three unique API keys for three different API consumers.
- Provide a mechanism for each consumer to submit their key through the X-API-Key HTTP request header.
- Validate the supplied key before processing the request.
- Return the result for a valid key and reject requests containing an invalid or missing key.
- Share the three keys securely with the assigned testers, who must test the API using cURL.
- Demonstrate that each valid key works and that an invalid key is rejected.

Security note: For this classroom exercise, keys may be generated using Python's `secrets` module. Do not commit real keys to a public repository or expose them in screenshots shared publicly.

Task 3: Authentication and Authorization in a Flask API [1 Mark]

Scenario: A company is developing an employee management API. The company wants only authenticated users to access the system and wants to restrict sensitive operations to authorized administrators.

Requirements:

- Develop a Flask API with endpoints for viewing, adding, updating, and deleting employee records.
- Implement authentication so a user must log in with valid credentials before accessing protected endpoints.
- Implement authorization using at least two roles:
    - Employee: can view employee records.
    - Administrator: can view, add, update, and delete employee records.
- Protect the endpoints so unauthenticated users cannot access them.
- Ensure that authenticated employees cannot perform administrator-only operations.
- Test the API using cURL with valid and invalid credentials and demonstrate the difference between authentication and authorization.